

Planning

⚙️ Status	In progress
📅 Due date	02/03/2025
🕒 Priority	High
🛠️ Task type	🎨 Polish
⌚ Effort level	Medium
≡ Description	plaining how to do the project

Pick your rendering approach first

You need a library to talk to the GPU, since raw Java has no 3D rendering:

- **LWJGL (Lightweight Java Game Library)** — the standard choice, gives you raw OpenGL bindings. More control, steeper learning curve.
- **jMonkeyEngine** — a full game engine built on top of LWJGL, has built-in terrain tools (`TerrainQuad` , hydraulic erosion, etc.) if you want less boilerplate.
- **LibGDX** — another popular Java game framework, has terrain examples in its ecosystem too.

Given you've already been digging into things like Crow/Pistache for C++ HTTP layers, I'd guess you enjoy understanding the plumbing — so LWJGL raw OpenGL might suit you better than a black-box engine.

Repos worth studying

kosa12/proc_terrain_3D_gen is a 3D procedural terrain generator in Java using LWJGL and OpenGL, with real-time ImGui controls, Perlin noise with fractal Brownian motion, biomes, texture atlasing, and chunk-based rendering. It even supports multiple noise types — Standard for smooth hills, Ridged for mountains, Billowy for rolling hills, and Hybrid combining the two. This is probably your best single reference repo — recent, well-scoped, and uses JDK 21 + Gradle.

[GitHub](#)[GitHub](#)

Terasology (MovingBlocks/Terasology) — an open source project started to research procedural terrain generation and efficient rendering techniques in Java using LWJGL, using a block-based voxel approach similar to Minecraft. Bigger and more mature codebase — good for seeing how a real team structures a voxel terrain engine, but heavier to digest. [github](#)

Also browse [github.com/topics/terrain-generation?l=java](#) — filtered to Java repos, several smaller heightmap/Perlin-noise projects there worth skimming for ideas.

Videos

- **ThinMatrix's "OpenGL 3D Game Tutorial" series** (LWJGL-based) — episode 37 specifically covers procedural terrain generation. This series is the most commonly recommended starting point for Java+OpenGL terrain work.
- **"Creating a Game Engine in Java and OpenGL" series**, episode 4 covers heightmap-based terrain generation with LWJGL 3.

Core algorithms to actually research

Regardless of engine, the generation logic is engine-agnostic — this is where most of your learning will happen:

1. **Perlin / Simplex noise** — smooth, natural-looking heightmaps, the industry standard starting point
2. **Fractal Brownian Motion (fBm)** — layering multiple octaves of noise for realistic detail at different scales
3. **Diamond-Square algorithm** — an alternative heightmap technique, simpler to implement than Perlin but has visible grid artifacts
4. **Marching Cubes** — if you want voxel/cave-capable terrain rather than just a heightmap
5. **Erosion simulation** (thermal/hydraulic) — the step that takes generated noise from "video-gamey" to "looks like a real landscape"

Suggested path given your background

Since you like building things incrementally and validating step by step (like you did with the Zebra scanner project):

1. Get LWJGL rendering a flat colored grid/mesh
2. Swap flat heights for Perlin-noise-generated heights
3. Add camera movement (WASD, like you did in past projects)
4. Layer in fBm for more natural terrain
5. Add texturing/biomes based on height thresholds
6. Optional: chunk-based loading for larger worlds

Here's the practical path — order matters, since each builds on the last.

1. LWJGL setup — start here

The official configuration tool builds your Gradle/Maven file for you (pick modules: core + GLFW + OpenGL + JOGL preset is even in the dropdown): <https://www.lwjgl.org/customize> — then follow the official "Get Started" guide for the actual HelloWorld window: <https://www.lwjgl.org/guide>

Once you have a blank window opening, the community-written **"3D Game Development with LWJGL 3" book** is the most complete free tutorial and covers exactly your pipeline order (window → shaders → mesh rendering → camera → later chapters get into terrain-adjacent topics): <https://lwjglgamedev.gitbooks.io/3d-game-development-with-lwjgl/content/>

2. Video: ThinMatrix's OpenGL 3D Game Tutorial series

Still the standard recommendation for LWJGL specifically. Watch in order — you genuinely need the early episodes (window, shaders, loading a mesh, camera

movement) before terrain makes sense. Episode ~37 covers procedural terrain and is the payoff, but skipping to it without the setup episodes won't work since it assumes you already have a render pipeline.

3. JOML

Small library, so the README basically is the docs —

shows `Matrix4f`, `Vector3f` usage inline: <https://github.com/JOML-CI/JOML>

You'll mainly reference this when writing your camera/projection matrix code, not something you read cover to cover.

4. FastNoiseLite — Java usage

The Java-specific getting started page is short. The actual pattern is basically this:

java

```
FastNoiseLite noise = new FastNoiseLite(); noise.SetNoiseType(
FastNoiseLite.NoiseType.Perlin); noise.SetFractalType(FastNoiseLite.FractalType.FBm); float[][] heightmap = new float[128][128]; for (int x = 0; x < 128; x++) { for (int z = 0; z < 128; z++) { heightmap[x][z] = noise.GetNoise(x, z); } }
```

Full Java folder with the source + a

README: <https://github.com/Auburn/FastNoiseLite/tree/master/Java>

Also genuinely useful before writing any code: the **web preview app** lets you tweak noise type/octaves/gain sliders and see the result live, so you can pick settings visually before committing them to code: <https://auburn.github.io/FastNoiseLite/>

Suggested order for your team:

1. LWJGL guide → get a window open (30 min)
2. ThinMatrix ep 1—a few → get a colored triangle/mesh rendering with a moving camera
3. Play with FastNoiseLite's web preview app → pick noise settings

4. Write the noise-to-heightmap loop (just Java, no rendering yet — print/dump it to confirm values look right)
5. Feed that heightmap into the mesh-building loop from step 2, using JOML for normals

Want me to write out that mesh-building loop (heightmap array → vertex/index buffers) as actual code once you've got a window rendering?